Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО–КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
дисциплины
«Основы нейронных сетей»
Вариант 10

Выполнил:
Рябинин Егор Алексеевич
3 курс, группа ИВТ-б-о-23-2,
09.03.01 «Информатика и вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств вычислительной техники и автоматизированных систем», очная форма обучения

_____
(подпись)

Проверил:
Доцент департамента цифровых, робототехнических систем и электроники института перспективной инженерии
Воронкин Роман Александрович
_____
(подпись)

Отчет защищен с оценкой _____ Дата защиты_____

Ставрополь, 2026 г

Тема: основы построения и обучения нейросетевых моделей в фреймворках TensorFlow и PyTorch.

Цель: изучение архитектуры простых нейронных сетей и приобретение практических навыков их программной реализации, обучения и подбора гиперпараметров (функций активации, оптимизаторов) для решения задач анализа данных.

Порядок выполнения работы:

Задание 1. Создайте систему компьютерного зрения, которая будет определять тип геометрической фигуры. Используя подготовленную базу и шаблон ноутбука проведите серию экспериментов по перебору гиперпараметров нейронной сети, распознающей три категории изображений (треугольник, круг, квадрат).

1. Поменяйте количество нейронов в сети, используя следующие значения:

– один слой 10 нейронов;

– один слой 100 нейронов;

– один слой 5000 нейронов.

2. Поменяйте активационную функцию в скрытых слоях с «relu» на «linear».

3. Поменяйте размеры batch_size:

– 10;

– 100;

– 1000.

4. Выведите на экран получившиеся точности.

Всего должно получиться 18 комбинаций указанных параметров.

Создайте сравнительную таблицу по результатам проведенных тестов.

1.1. Подключение необходимых библиотек.

Подключаются все необходимые библиотеки: NumPy для работы с массивами, Matplotlib для визуализации, Pandas для создания таблиц результатов. Из фреймворка PyTorch импортируются модули для построения

нейросетей (nn), оптимизаторы (optim) и инструменты для работы с данными (DataLoader, TensorDataset). Библиотеки PIL и torchvision нужны для загрузки и обработки изображений. Отключены предупреждения для чистоты вывода.

```
# Подключение библиотек
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import os
import warnings
warnings.filterwarnings('ignore')

# PyTorch
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

# Для обработки изображений
from torchvision import transforms
from PIL import Image

# Для отрисовки
%matplotlib inline
```

Рисунок 1.1 – Подключение необходимых библиотек

1.2. Загрузка данных с Google Drive.

С помощью библиотеки gdown выполняется загрузка архива с данными из Google Drive по указанной ссылке. После скачивания архив распаковывается в текущую директорию. Внутри находятся изображения геометрических фигур (треугольников, кругов, квадратов), разложенные по папкам в соответствии с классом.

```
import gdown
url = 'https://drive.google.com/uc?id=1yt_hr4AYEWbjtmYfTNSlEkxl5JaFuYEX'
gdown.download(url, 'hw_light.zip', quiet=False)
!unzip -o -q hw_light.zip

print("Данные загружены")
```

Рисунок 1.2 – Загрузка данных с Google Drive

1.3. Чтение и предобработка изображений.

Выполняется загрузка изображений из распакованной директории. Для каждой папки (класса) читаются все файлы изображений. Каждое изображение конвертируется в оттенки серого и изменяет размер до 20x20 пикселей для унификации входных данных. Преобразованные изображения сохраняются в список x_train. Параллельно формируется список меток y_train: папка 0 соответствует треугольнику (метка 0), папка 3 – кругу (метка 1), остальные – квадрату (метка 2). Всего загружено 302 изображения размером 20x20 пикселей.

```
base_dir = '/content/hw_light'
x_train, y_train = [], []
img_height, img_width = 20, 20

print("Загрузка изображений...")
for patch in os.listdir(base_dir):
    patch_path = os.path.join(base_dir, patch)
    if os.path.isdir(patch_path):
        for img_file in os.listdir(patch_path):
            img_path = os.path.join(patch_path, img_file)
            img = Image.open(img_path).convert('L')
            img = img.resize((img_height, img_width))
            img_array = np.array(img).astype('float32')
            x_train.append(img_array)

            if patch == '0':
                y_train.append(0)
            elif patch == '3':
                y_train.append(1)
            else:
                y_train.append(2)

x_train = np.array(x_train)
y_train = np.array(y_train)

print(f'Изображений: {x_train.shape}')
print(f'Меток: {y_train.shape}')
```

[18]

```
Загрузка изображений...
Изображений: (302, 20, 20)
Меток: (302,)
```

Рисунок 1.3 – Чтение и предобработка изображений

1.4. Подготовка данных для нейросети.

Данные подготавливаются для подачи в нейросеть. Изображения размером 20x20 преобразуются в одномерные вектора длиной 400 (reshape), так как полносвязная сеть принимает на вход плоский вектор. Затем выполняется нормализация пикселей делением на 255 – значения приводятся к диапазону [0, 1], что улучшает сходимость обучения. После этого данные конвертируются в тензоры PyTorch: FloatTensor для признаков и LongTensor для меток классов.

```
CLASS_COUNT = 3

x_train = x_train.reshape(x_train.shape[0], -1)
print(f'После reshape: {x_train.shape}')

x_train = x_train / 255.0
print(f'Нормализация: [{x_train.min():.2f}, {x_train.max():.2f}]')

x_tensor = torch.FloatTensor(x_train)
y_tensor = torch.LongTensor(y_train)

print(f'Тензоры: x {x_tensor.shape}, y {y_tensor.shape}')
```

```
После reshape: (302, 400)
Нормализация: [0.00, 1.00]
Тензоры: x torch.Size([302, 400]), y torch.Size([302])
```

Рисунок 1.4 – Подготовка данных для нейросети

1.5. Разделение данных на обучающую и валидационную выборки.

Данные разделяются на обучающую и валидационную выборки. Устанавливается seed (42) для воспроизводимости результатов. С помощью permutation данные случайным перемешиваются, чтобы исключить влияние порядка на обучение. 20% данных (60 изображений) отводится для валидации, остальные 80% (242 изображения) – для обучения. Валидационная выборка используется для оценки качества модели на данных, которые она не видела в процессе обучения.

```
np.random.seed(42)
indices = np.random.permutation(len(x_train))
x_tensor = x_tensor[indices]
y_tensor = y_tensor[indices]

val_size = int(0.2 * len(x_tensor))

x_train_split = x_tensor[val_size:]
y_train_split = y_tensor[val_size:]

x_val = x_tensor[:val_size]
y_val = y_tensor[:val_size]

print(f'Обучающая: {x_train_split.shape}')
print(f'Валидационная: {x_val.shape}')
```

```
Обучающая: torch.Size([242, 400])
Валидационная: torch.Size([60, 400])
```

Рисунок 1.5 – Разделение данных на обучающую и валидационную выборки

1.6. Определение архитектуры нейронной сети.

Создается класс нейронной сети с одним скрытым слоем. В конструкторе задаются параметры: размер входа (400 пикселей), количество нейронов в скрытом слое, тип активации и число классов на выходе. В зависимости от выбранной активации используется ReLU или линейная функция (Identity). Метод forward определяет проход данных: входной вектор подается на скрытый слой, затем применяется активация, после чего результат передается на выходной слой с тремя нейронами (по числу классов).

```python
class OneLayerNN(nn.Module):
    def __init__(self, input_dim=400, hidden_neurons=10, activation='relu', num_classes=3):
        super(OneLayerNN, self).__init__()

        if activation == 'relu':
            self.activation = nn.ReLU()
        else:
            self.activation = nn.Identity()

        self.hidden = nn.Linear(input_dim, hidden_neurons)
        self.output = nn.Linear(hidden_neurons, num_classes)

    def forward(self, x):
        x = self.hidden(x)
        x = self.activation(x)
        x = self.output(x)
        return x
```

Рисунок 1.6 – Определение архитектуры нейронной сети

1.7. Функция для обучения и оценки модели.

Функция реализует процесс обучения модели с заданными гиперпараметрами. Создается модель с указанным числом нейронов и типом активации, определяется функция потерь CrossEntropyLoss (подходит для многоклассовой классификации) и оптимизатор Adam. Данные загружаются батчами указанного размера. В каждой эпохе модель обучается на батчах из обучающей выборки, затем вычисляется точность на валидационной выборке. Функция возвращает итоговую точность после 50 эпох обучения.

```python
def train_and_evaluate_pytorch(neurons, activation, batch_size, epochs=50):
    model = OneLayerNN(input_dim=400, hidden_neurons=neurons,
                       activation=activation, num_classes=CLASS_COUNT)
    model = model.to(device)

    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    train_dataset = TensorDataset(x_train_split, y_train_split)
    val_dataset = TensorDataset(x_val, y_val)

    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
    val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)

    best_val_acc = 0.0

    for epoch in range(epochs):
        model.train()
        for inputs, labels in train_loader:
            inputs, labels = inputs.to(device), labels.to(device)

            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

        model.eval()
        correct = 0
        total = 0
        with torch.no_grad():
            for inputs, labels in val_loader:
                inputs, labels = inputs.to(device), labels.to(device)
                outputs = model(inputs)
                _, predicted = torch.max(outputs.data, 1)
                total += labels.size(0)
                correct += (predicted == labels).sum().item()

        val_acc = correct / total
        if val_acc > best_val_acc:
            best_val_acc = val_acc

    return best_val_acc
```

Рисунок 1.7 – Функция для обучения и оценки модели

1.8. Перебор гиперпараметров.

Выполняется полный перебор всех комбинаций гиперпараметров (18 вариантов). Для каждого сочетания количества нейронов (10, 100, 5000), типа активации (ReLU, linear) и размера батча (10, 100, 1000) запускается

обучение модели. Результаты (точность на валидации) сохраняются в список для последующего анализа. При возникновении ошибок точность фиксируется как 0.0.

```
neurons_list = [10, 100, 5000]
activations = ['relu', 'linear']
batch_sizes = [10, 100, 1000]

results = []

total = len(neurons_list) * len(activations) * len(batch_sizes)
current = 0

for neurons in neurons_list:
    for activation in activations:
        for batch_size in batch_sizes:
            current += 1
            print(f"\n[{current}/{total}] {neurons} нейр., {activation}, batch={batch_size}")

            try:
                accuracy = train_and_evaluate_pytorch(neurons, activation, batch_size, epochs=50)
                results.append({
                    'Нейронов': neurons,
                    'Активация': activation,
                    'Batch Size': batch_size,
                    'Точность': round(accuracy, 4)
                })
                print(f"✓ Точность: {accuracy:.4f}")
            except Exception as e:
                print(f"✗ Ошибка: {e}")
                results.append({
                    'Нейронов': neurons,
                    'Активация': activation,
                    'Batch Size': batch_size,
                    'Точность': 0.0
                })
```

Рисунок 1.7 – Перебор гиперпараметров

```
[1/18] 10 нейр., relu, batch=10
✓ Точность: 0.8000

[2/18] 10 нейр., relu, batch=100
✓ Точность: 0.7833

[3/18] 10 нейр., relu, batch=1000
✓ Точность: 0.7000

[4/18] 10 нейр., linear, batch=10
✓ Точность: 0.7833

[5/18] 10 нейр., linear, batch=100
✓ Точность: 0.7667

[6/18] 10 нейр., linear, batch=1000
✓ Точность: 0.7333

[7/18] 100 нейр., relu, batch=10
✓ Точность: 0.8500

[8/18] 100 нейр., relu, batch=100
✓ Точность: 0.8333
...
✓ Точность: 0.7500

[18/18] 5000 нейр., linear, batch=1000
✓ Точность: 0.7500
```

Рисунок 1.8 – Перебор гиперпараметров

1.9. Создание сравнительной таблицы результатов.

```
df = pd.DataFrame(results)
df_sorted = df.sort_values('Точность', ascending=False).reset_index(drop=True)

print("СРАВНИТЕЛЬНАЯ ТАБЛИЦА РЕЗУЛЬТАТОВ (PyTorch)")
print("=" * 80)
print(df_sorted.to_string(index=True))
print("=" * 80)

df_sorted.to_csv('experiment_results_pytorch.csv', index=False)
print("\nТаблица сохранена")

pivot = df.pivot_table(values='Точность',
                       index=['Нейронов', 'Активация'],
                       columns='Batch Size',
                       aggfunc='mean').round(4)
print("\nСводная таблица:")
print(pivot)
```

Рисунок 1.9 – Создание сравнительной таблицы результатов.

```
СРАВНИТЕЛЬНАЯ ТАБЛИЦА РЕЗУЛЬТАТОВ (PyTorch)
================================================================================
    Нейронов Активация  Batch Size  Точность
0       5000      relu         100    0.8667
1       5000      relu        1000    0.8667
2        100      relu          10    0.8500
3       5000      relu          10    0.8500
4        100      relu         100    0.8333
5        100      relu        1000    0.8167
6         10      relu          10    0.8000
7         10      relu         100    0.7833
8        100    linear         100    0.7833
9         10    linear          10    0.7833
10       100    linear        1000    0.7667
11        10    linear         100    0.7667
12       100    linear          10    0.7667
13      5000    linear          10    0.7500
14      5000    linear         100    0.7500
15      5000    linear        1000    0.7500
16        10    linear        1000    0.7333
17        10      relu        1000    0.7000
================================================================================

Таблица сохранена
...
100      linear     0.7667  0.7833  0.7667
         relu       0.8500  0.8333  0.8167
5000     linear     0.7500  0.7500  0.7500
         relu       0.8500  0.8667  0.8667
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Рисунок 1.10 – Сравнительная таблица результатов

1.10. Выбор лучшей модели и ее обучение.

Из таблицы результатов выбирается комбинация гиперпараметров, показавшая наилучшую точность. Создается модель с этими параметрами и выполняется ее финальное обучение в течении 30 эпох. Используются те же функция потерь и оптимизатор, что и при переборе. Обученная модель будет использована для итогового тестирования и визуализации предсказаний.

```
best_params = df_sorted.iloc[0]
print(f"Лучшая комбинация:")
print(f"   Нейронов: {best_params['Нейронов']}")
print(f"   Активация: {best_params['Активация']}")
print(f"   Batch Size: {best_params['Batch Size']}")
print(f"   Точность: {best_params['Точность']:.4f}")

best_model = OneLayerNN(input_dim=400,
                        hidden_neurons=int(best_params['Нейронов']),
                        activation=best_params['Активация'],
                        num_classes=CLASS_COUNT)
best_model = best_model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(best_model.parameters(), lr=0.001)

train_dataset = TensorDataset(x_train_split, y_train_split)
train_loader = DataLoader(train_dataset, batch_size=int(best_params['Batch Size']), shuffle=True)
val_dataset = TensorDataset(x_val, y_val)
val_loader = DataLoader(val_dataset, batch_size=int(best_params['Batch Size']), shuffle=False)

print("\nФинальное обучение лучшей модели...")
for epoch in range(30):
    best_model.train()
    for inputs, labels in train_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = best_model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

    if (epoch+1) % 5 == 0:
        best_model.eval()
        correct = 0
        total = 0
        with torch.no_grad():
            for inputs, labels in val_loader:
                inputs, labels = inputs.to(device), labels.to(device)
                outputs = best_model(inputs)
                _, predicted = torch.max(outputs.data, 1)
                total += labels.size(0)
                correct += (predicted == labels).sum().item()
        val_acc = correct / total
        print(f"Эпоха {epoch+1}, Точность: {val_acc:.4f}")
```

Рисунок 1.11 – Выбор лучшей модели и ее обучение

1.11. Оценка лучшей модели и визуализация предсказаний.

Выполняется финальная оценка лучшей модели на валидационной выборке. Подсчитывается общая точность классификации. Для наглядной демонстрации работы модели случайным образом выбираются 8 изображений, отображаются сами изображения, а также предсказанный и истинный классы. Правильные предсказания выделяются зеленым цветом,

ошибки – красным. Это позволяет визуально оценить, на каких изображениях модель ошибается.

```
best_model.eval()
correct = 0
total = 0
all_preds = []
all_labels = []

with torch.no_grad():
    for inputs, labels in val_loader:
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = best_model(inputs)
        _, predicted = torch.max(outputs.data, 1)
        total += labels.size(0)
        correct += (predicted == labels).sum().item()
        all_preds.extend(predicted.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

final_acc = correct / total
print(f"Итоговая точность на валидации: {final_acc:.4f} ({final_acc*100:.1f}%)")

plt.figure(figsize=(16, 4))
indices = np.random.choice(len(x_val), 8, replace=False)

for i, idx in enumerate(indices):
    img = x_val[idx].cpu().numpy().reshape(20, 20)
    true = all_labels[idx] if idx < len(all_labels) else 0
    pred = all_preds[idx] if idx < len(all_preds) else 0

    plt.subplot(1, 8, i+1)
    plt.imshow(img, cmap='gray')
    color = 'green' if pred == true else 'red'
    plt.title(f'П:{class_names[pred]}\nИ:{class_names[true]}', color=color, fontsize=9)
    plt.axis('off')

plt.suptitle(f'Предсказания (зеленый - верно). Общая точность: {final_acc:.3f}', fontsize=14)
plt.tight_layout()
plt.show()
```

Рисунок 1.12 – Оценка лучшей модели и визуализация предсказаний

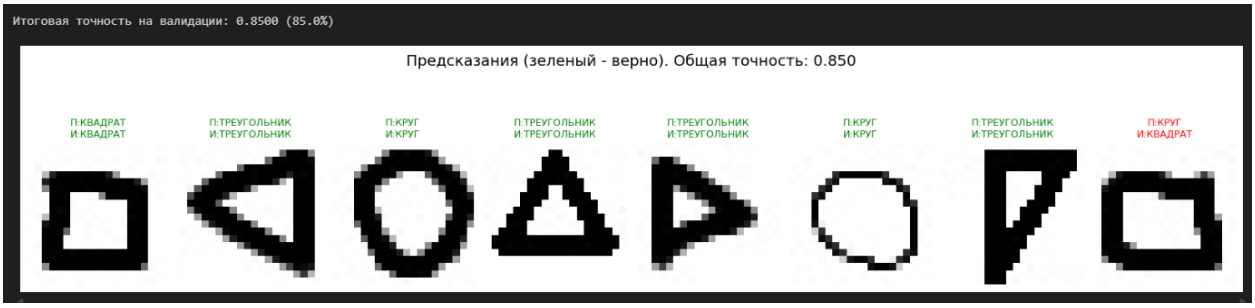

Рисунок 1.13 – Визуализация предсказаний

Задание 2. Выполнить задание 1, используя фреймворк TensorFlow.

2.1. Подключение необходимых библиотек.

Подключаются все необходимые библиотеки. NumPy используется для работы с массивами, Matplotlib для визуализации результатов, Pandas для создания таблиц. Из TensorFlow/Keras импортируются компоненты для построения нейронных сетей: Sequential для последовательной модели, Dense для полносвязных слоев, utils для one–hot кодирования, модуль image для загрузки изображений и оптимизатор Adam. Отключены предупреждения для чистоты вывода.

```
# Подключение библиотек
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import os
import warnings
warnings.filterwarnings('ignore')

# TensorFlow/Keras
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras import utils
from tensorflow.keras.preprocessing import image
from tensorflow.keras.optimizers import Adam

# Для отрисовки
%matplotlib inline
```

Рисунок 2.1 – Подключение необходимых библиотек

2.2. Загрузка данных с Google Drive.

С помощью библиотеки gdown выполняется загрузка архива с данными из Google Drive. После скачивания архив распаковывается. Внутри находятся изображения геометрических фигур (круги, треугольники, квадраты), разложенные по папкам в соответствии с классом.

```
# Загрузка датасета
import gdown
url = 'https://drive.google.com/uc?id=1yt_hr4AYEWbjtmYfTNSlEkxl5JaFuYEX'
gdown.download(url, 'hw_light.zip', quiet=False)
!unzip -o -q hw_light.zip

print("Данные загружены!")
```

```
Downloading...
From: https://drive.google.com/uc?id=1yt_hr4AYEWbjtmYfTNSlEkxl5JaFuYEX
To: /content/hw_light.zip
100%|██████████| 304k/304k [00:00<00:00, 65.3MB/s]
Данные загружены!
```

Рисунок 2.2 – Загрузка данных с Google Drive

2.3. Чтение и предобработка изображений.

Выполняется загрузка изображений из распакованной директории. Для каждой папки (класса) читаются все файлы изображений. Используется встроенная функция Keras image.load_img, которая загружает изображение, сразу изменяет его размер до 20x20 пикселей и конвертирует в оттенки серого. Затем изображение преобразуется в массив NumPy с помощью img_to_array. Параллельно формируется список меток: папка 0 соответствует кругу (метка 0), папка 3 – треугольнику (метка 1), остальные папки – квадрату (метка 2). Всего загружено 302 изображения размером 20x20x1 (с одним каналом).

```python
base_dir = '/content/hw_light'
x_train, y_train = [], []
img_height, img_width = 20, 20

print("Загрузка изображений...")
for patch in os.listdir(base_dir):
    patch_path = os.path.join(base_dir, patch)
    if os.path.isdir(patch_path):
        for img_file in os.listdir(patch_path):
            img = image.load_img(
                os.path.join(patch_path, img_file),
                target_size=(img_height, img_width),
                color_mode='grayscale'
            )
            x_train.append(image.img_to_array(img))

            if patch == '0':
                y_train.append(0)  # 0 = КРУГ
            elif patch == '3':
                y_train.append(1)  # 1 = ТРЕУГОЛЬНИК
            else:  # patch == '1'
                y_train.append(2)  # 2 = КВАДРАТ

x_train = np.array(x_train)
y_train = np.array(y_train)

print(f'Изображений: {x_train.shape}')
print(f'Меток: {y_train.shape}')
```

[3]

```
Загрузка изображений...
Изображений: (302, 20, 20, 1)
Меток: (302,)
```

Рисунок 2.3 – Чтение и предобработка изображений

2.4. Нормализация и one–hot кодирование.

Данные подготавливаются для подачи в нейросеть. Изображения размером 20x20x1 преобразуются в одномерные вектора длиной 400 с помощью reshape. Выполняется нормализация пикселей делением на 255 – значения приводятся к диапазону [0, 1], что ускоряет сходимость обучения. Метки классов преобразуются в one–hot формат с помощью to_categorical: например, метка 0 превращается в вектор [1, 0, 0], метка 1 – [0, 1, 0] и т.д. Это необходимо для использования функции потерь categorical_crossentropy.

```python
CLASS_COUNT = 3

# Преобразование в вектор
x_train = x_train.reshape(x_train.shape[0], -1)
print(f'После reshape: {x_train.shape}')

# Нормализация
x_train = x_train.astype('float32') / 255.
print(f'Нормализация: [{x_train.min():.2f}, {x_train.max():.2f}]')

# One-hot encoding
y_train_cat = utils.to_categorical(y_train, CLASS_COUNT)
print(f'One-hot: {y_train_cat.shape}')
```

[4]

```
После reshape: (302, 400)
Нормализация: [0.00, 1.00]
One-hot: (302, 3)
```

Рисунок 2.4 – Нормализация и one–hot кодирование

2.5. Перемешивание и разделение данных.

Данные перемешиваются и разделяются на обучающую и валидационную выборки. Устанавливается seed (42) для воспроизводимости результатов. С помощью permutation данные случайным образом перемешиваются, чтобы исключить влияние порядка на обучение. 20% данных (60 изображений) отводится для валидации, остальные 80% (242 изображения) – для обучения. Валидационная выборка используется для оценки качества модели на данных, которые она не видела..

```python
# Перемешивание
np.random.seed(42)
indices = np.random.permutation(len(x_train))
x_train = x_train[indices]
y_train_cat = y_train_cat[indices]

# Разделение 80/20
val_size = int(0.2 * len(x_train))

x_train_split = x_train[val_size:]
y_train_split = y_train_cat[val_size:]

x_val = x_train[:val_size]
y_val = y_train_cat[:val_size]

print(f'Обучающая: {x_train_split.shape}')
print(f'Валидационная: {x_val.shape}')
```

```
Обучающая: (242, 400)
Валидационная: (60, 400)
```

Рисунок 2.5 – Перемешивание и разделение данных

2.6. Определение архитектуры нейронной сети.

Функция создает модель нейронной сети с одним скрытым слоем. Используется Sequential API Keras – последовательная модель, где слои добавляются один за другим. Первый слой (скрытый) – полносвязный (Dense) с количеством нейронов, заданным параметром neurons. Он принимает входные данные размерностью 400 и использует указанную функцию активации (ReLU или linear). Выходной слой содержит 3 нейрона (по числу классов) с функцией активации softmax, которая преобразует выходы в вероятности принадлежности к каждому классу.

```python
def create_model(neurons, activation='relu'):
    model = Sequential()
    model.add(Dense(neurons,
                    input_dim=400,
                    activation=activation,
                    name='hidden_layer'))
    model.add(Dense(CLASS_COUNT,
                    activation='softmax',
                    name='output'))
    return model
```

Рисунок 2.6 – Определение архитектуры нейронной сети

2.7. Функция для обучения и оценки модели.

Функция реализует процесс обучения модели с заданными гиперпараметрами. Сначала создается модель с указанным числом нейронов и типом активации. Модель компилируется с функцией потерь categorical_crossentropy (подходит для многоклассовой классификации с one–hot метками), оптимизатором Adam и метрикой accuracy. Затем выполняется обучение на указанное количество эпох с заданным размером батча. Обучение происходит без вывода промежуточных результатов (verbose=0). Функция возвращает максимальную точность на валидационной выборке за все эпохи обучения.

```python
def train_and_evaluate(neurons, activation, batch_size, epochs=50):
    model = create_model(neurons, activation)

    model.compile(loss='categorical_crossentropy',
                  optimizer=Adam(learning_rate=0.001),
                  metrics=['accuracy'])

    history = model.fit(x_train_split, y_train_split,
                        batch_size=batch_size,
                        epochs=epochs,
                        validation_data=(x_val, y_val),
                        verbose=0)

    best_acc = max(history.history['val_accuracy'])
    return best_acc
```

Рисунок 2.7 – Функция для обучения и оценки модели

2.8. Перебор гиперпараметров.

Выполняется полный перебор всех комбинаций гиперпараметров (18 вариантов). Для каждого сочетания количества нейронов (10, 100, 5000), типа активации (ReLU, linear) и размера батча (10, 100, 1000) запускается обучение модели с помощью функции train_and_evaluate. Результаты (точность на валидации) сохраняются в список results для последующего анализа. При возникновении ошибок точность фиксируется как 0.0.

```
# Параметры для перебора
neurons_list = [10, 100, 5000]
activations = ['relu', 'linear']
batch_sizes = [10, 100, 1000]

results = []

total = len(neurons_list) * len(activations) * len(batch_sizes)
current = 0

for neurons in neurons_list:
    for activation in activations:
        for batch_size in batch_sizes:
            current += 1
            print(f"\n[{current}/{total}] {neurons} нейр., {activation}, batch={batch_size}")

            try:
                accuracy = train_and_evaluate(neurons, activation, batch_size, epochs=50)
                results.append({
                    'Нейронов': neurons,
                    'Активация': activation,
                    'Batch Size': batch_size,
                    'Точность': round(accuracy, 4)
                })
                print(f"✓ Точность: {accuracy:.4f}")
            except Exception as e:
                print(f"X Ошибка: {e}")
                results.append({
                    'Нейронов': neurons,
                    'Активация': activation,
                    'Batch Size': batch_size,
                    'Точность': 0.0
                })
```

Рисунок 2.8 – Перебор гиперпараметров

```
[1/18] 10 нейр., relu, batch=10
✓ Точность: 0.8000

[2/18] 10 нейр., relu, batch=100
✓ Точность: 0.8000

[3/18] 10 нейр., relu, batch=1000
✓ Точность: 0.7333

[4/18] 10 нейр., linear, batch=10
✓ Точность: 0.7667

[5/18] 10 нейр., linear, batch=100
✓ Точность: 0.7333

[6/18] 10 нейр., linear, batch=1000
✓ Точность: 0.6000

[7/18] 100 нейр., relu, batch=10
✓ Точность: 0.8500

[8/18] 100 нейр., relu, batch=100
✓ Точность: 0.8167
...
✓ Точность: 0.7667

[18/18] 5000 нейр., linear, batch=1000
✓ Точность: 0.7500
Output is truncated. View as a scrollable element or open in
```

Рисунок 2.9 – Перебор гиперпараметров

2.9. Создание сравнительной таблицы результатов.

```python
# Создание DataFrame
df = pd.DataFrame(results)

# Сортировка по точности
df_sorted = df.sort_values('Точность', ascending=False).reset_index(drop=True)

print("СРАВНИТЕЛЬНАЯ ТАБЛИЦА РЕЗУЛЬТАТОВ")
print("=" * 80)
print(df_sorted.to_string(index=True))
print("=" * 80)

# Сохранение
df_sorted.to_csv('experiment_results_correct.csv', index=False)
print("\nТаблица сохранена")

# Сводная таблица
pivot = df.pivot_table(values='Точность',
                       index=['Нейронов', 'Активация'],
                       columns='Batch Size',
                       aggfunc='mean').round(4)
print("\nСводная таблица:")
print(pivot)
```

Рисунок 2.10 – Создание сравнительной таблицы результатов

2.10. Выбор лучшей модели и ее обучение.

Из таблицы результатов выбирается комбинация гиперпараметров, показавшая наилучшую точность. Создается модель с этими параметрами и выполняется ее финальное обучение в течение 50 эпох с выводом прогресса (verbose=1). После обучения веса модели сохраняются в файл для возможного последующего использования. Также задаются имена классов для подписей при визуализации.

```python
best_params = df_sorted.iloc[0]
print(f"Лучшая комбинация:")
print(f"   Нейронов: {best_params['Нейронов']}")
print(f"   Активация: {best_params['Активация']}")
print(f"   Batch Size: {best_params['Batch Size']}")
print(f"   Точность: {best_params['Точность']:.4f}")

best_model = create_model(int(best_params['Нейронов']), best_params['Активация'])
best_model.compile(loss='categorical_crossentropy',
                   optimizer=Adam(learning_rate=0.001),
                   metrics=['accuracy'])

history = best_model.fit(x_train_split, y_train_split,
                         batch_size=int(best_params['Batch Size']),
                         epochs=50,
                         validation_data=(x_val, y_val),
                         verbose=1)

best_model.save_weights('best_model_correct.weights.h5')
class_names = ['КРУГ', 'ТРЕУГОЛЬНИК', 'КВАДРАТ']
```

Рисунок 2.11 – Выбор лучшей модели и ее обучение

```
Лучшая комбинация:
   Нейронов: 5000
   Активация: relu
   Batch Size: 100
   Точность: 0.8667
Epoch 1/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 2s 464ms/step - accuracy: 0.3436 - loss: 2.7610 - val_accuracy: 0.5333 - val_loss: 1.3686
Epoch 2/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.4956 - loss: 1.8221 - val_accuracy: 0.6667 - val_loss: 0.7763
Epoch 3/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.6934 - loss: 0.8834 - val_accuracy: 0.5500 - val_loss: 1.3587
Epoch 4/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.6472 - loss: 1.4295 - val_accuracy: 0.6500 - val_loss: 0.8097
Epoch 5/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.7921 - loss: 0.7392 - val_accuracy: 0.5167 - val_loss: 0.9504
Epoch 6/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 0.5805 - loss: 0.8325 - val_accuracy: 0.7167 - val_loss: 0.7894
Epoch 7/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.7082 - loss: 0.6339 - val_accuracy: 0.7000 - val_loss: 0.6604
Epoch 8/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.8054 - loss: 0.6029 - val_accuracy: 0.6833 - val_loss: 0.6499
Epoch 9/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.8074 - loss: 0.5447 - val_accuracy: 0.7500 - val_loss: 0.6321
Epoch 10/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.8253 - loss: 0.4850 - val_accuracy: 0.7500 - val_loss: 0.6268
...
Epoch 49/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 1.0000 - loss: 0.0180 - val_accuracy: 0.8333 - val_loss: 0.5911
Epoch 50/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 1.0000 - loss: 0.0167 - val_accuracy: 0.8667 - val_loss: 0.6032
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Рисунок 2.12 – Выбор лучшей модели и ее обучение

2.11. Оценка лучшей модели и визуализация предсказаний.

Выполняется финальная оценка лучшей модели на валидационной выборке. Подсчитывается итоговая точность классификации. Для наглядной

демонстрации работы модели случайным образом выбираются 8 изображений из валидационной выборки. Для каждого изображения отображается само изображение, а также предсказанный класс (сверху) и истинный класс (снизу). Правильные предсказания выделяются зеленым цветом, ошибки – красным. Визуализация позволяет оценить, на каких изображениях модель ошибается и насколько хорошо она справляется с задачей в целом.

```
# Оценка на валидации
val_loss, val_acc = best_model.evaluate(x_val, y_val, verbose=0)
print(f"Итоговая точность на валидации: {val_acc:.4f} ({val_acc*100:.1f}%)")

# Покажем несколько предсказаний
plt.figure(figsize=(16, 4))
for i in range(8):
    idx = np.random.randint(len(x_val))
    img = x_val[idx].reshape(20, 20)
    true = np.argmax(y_val[idx])
    pred = np.argmax(best_model.predict(x_val[idx].reshape(1, -1), verbose=0)[0])

    plt.subplot(1, 8, i+1)
    plt.imshow(img, cmap='gray')
    color = 'green' if pred == true else 'red'
    plt.title(f'П:{class_names[pred]}\nИ:{class_names[true]}', color=color, fontsize=9)
    plt.axis('off')
plt.suptitle(f'Предсказания (зеленый - верно). Общая точность: {val_acc:.3f}', fontsize=14)
plt.tight_layout()
plt.show()
```

Рисунок 2.13 – Оценка лучшей модели и визуализация предсказаний

Рисунок 2.14 – Визуализация предсказаний

Задание 3. Самостоятельно напишите нейронную сеть, которая может стать составной частью системы бота для игры в «Крестики–нолики». Используя подготовленную базу изображений, создайте и обучите нейронную сеть, распознающую две категории изображений: крестики и нолики. Добейтесь точности распознавания более 95% (accuracy).

3.1. Подключение необходимых библиотек.

Подключаются все необходимые библиотеки. Из TensorFlow/Keras импортируются компоненты для построения нейронной сети: Sequential для последовательной модели, Dense для полносвязных слоев, Dropout для регуляризации (борьбы с переобучением), utils для one–hot кодирования, модуль image для загрузки изображений, оптимизатор Adam и callback EarlyStopping для ранней остановки обучения. Отключены предупреждения для чистоты вывода.

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import os
import warnings
warnings.filterwarnings('ignore')

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras import utils
from tensorflow.keras.preprocessing import image
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping

%matplotlib inline
```

Рисунок 3.1 – Подключение необходимых библиотек

3.2. Загрузка данных с Google Drive.

С помощью библиотеки gdown выполняется загрузка архива с данными из Google Drive. После скачивания архив распаковывается. Внутри находятся изображения крестиков и ноликов, разложенные по папкам в соответствии с классом.

```
import gdown
url = 'https://drive.google.com/uc?id=1STlK6jwSU7NohjMlYBnlpiGlHh_N5fia'
gdown.download(url, 'hw_pro.zip', quiet=False)
!unzip -o -q hw_pro.zip
print("Готово")
```

```
Downloading...
From: https://drive.google.com/uc?id=1STlK6jwSU7NohjMlYBnlpiGlHh_N5fia
To: /content/hw_pro.zip
100%|██████████| 101k/101k [00:00<00:00, 3.33MB/s]
Готово
```

Рисунок 3.2 – Загрузка данных с Google Drive

3.3. Чтение и предобработка изображений.

Выполняется загрузка изображений из распакованной директории. Для каждой папки читаются все файлы изображений с помощью функции Keras image.load_img, которая загружает изображение, изменяет размер до 20x20 пикселей и конвертирует в оттенки серого. Затем изображение преобразуется в массив NumPy. Параллельно формируется список меток: папка 0 содержит нолики (метка 1), остальные папки – крестики (метка 0). Всего загружено 102 изображения: 52 крестика и 50 ноликов – примерно сбалансированный набор данных.

```python
base_dir = '/content/hw_pro'
x_train, y_train = [], []
img_height, img_width = 20, 20

for patch in os.listdir(base_dir):
    patch_path = os.path.join(base_dir, patch)
    if os.path.isdir(patch_path):
        for img_file in os.listdir(patch_path):
            img = image.load_img(
                os.path.join(patch_path, img_file),
                target_size=(img_height, img_width),
                color_mode='grayscale'
            )
            x_train.append(image.img_to_array(img))

            if patch == '0':
                y_train.append(1)  # 1 = НОЛИК
            else:
                y_train.append(0)  # 0 = КРЕСТИК

x_train = np.array(x_train)
y_train = np.array(y_train)

print(f'Изображений: {x_train.shape}')
print(f'Меток: {y_train.shape}')
print(f'0 (КРЕСТИК): {np.sum(y_train==0)}')
print(f'1 (НОЛИК): {np.sum(y_train==1)}')
```

```
Изображений: (102, 20, 20, 1)
Меток: (102,)
0 (КРЕСТИК): 52
1 (НОЛИК): 50
```

Рисунок 3.3 – Чтение и предобработка изображений

3.4. Нормализация и one–hot кодирование.

Данные подготавливаются для подачи в нейросеть. Изображения размером 20x20x1 преобразуются в одномерные вектора длиной 400 с помощью reshape. Выполняется нормализация пикселей делением на 255 – значения приводятся к диапазону [0, 1], что ускоряет сходимость обучения. Метки классов преобразуются в one–hot формат: крестик (0) превращается в [1, 0], нолик (1) – в [0, 1]. Это необходимо для использования функции потерь categorical_crossentropy.

```
CLASS_COUNT = 2

x_train = x_train.reshape(x_train.shape[0], -1)
x_train = x_train.astype('float32') / 255.
y_train_cat = utils.to_categorical(y_train, CLASS_COUNT)

print(f'x: {x_train.shape}, y: {y_train_cat.shape}')
```

```
x: (102, 400), y: (102, 2)
```

Рисунок 3.4 – Нормализация и one–hot кодирование

3.5. Перемешивание и разделение данных.

Данные перемешиваются и разделяются на обучающую и валидационную выборки. Устанавливается seed (42) для воспроизводимости результатов. С помощью permutation данные случайным образом перемешиваются. 20% данных (20 изображений) отводится для валидации, остальные 80% (82 изображения) – для обучения. Валидационная выборка используется для оценки качества модели в процессе обучения и для ранней остановки.

```
np.random.seed(42)
indices = np.random.permutation(len(x_train))
x_train = x_train[indices]
y_train_cat = y_train_cat[indices]

val_size = int(0.2 * len(x_train))
x_val = x_train[:val_size]
y_val = y_train_cat[:val_size]
x_train_split = x_train[val_size:]
y_train_split = y_train_cat[val_size:]

print(f'Train: {x_train_split.shape}, Val: {x_val.shape}')
```

```
Train: (82, 400), Val: (20, 400)
```

Рисунок 3.5 – Перемешивание и разделение данных

3.6. Создание и обучение модели.

В этом блоке создается и обучается нейронная сеть для распознавания крестиков и ноликов. Модель имеет два скрытых слоя: первый с 54

нейронами и активацией ReLU, второй с 27 нейронами и активацией ReLU. После каждого скрытого слоя добавлен Dropout с вероятностью 0.5, который случайным образом отключает половину нейронов во время обучения для предотвращения переобучения. Выходной слой содержит 2 нейрона с активацией softmax для бинарной классификации. Модель компилируется с функцией потерь categorical_crossentropy, оптимизатором Adam и метрикой accuracy. Используется callback EarlyStopping, который отслеживает точность на валидационной выборке и останавливает обучение, если точность не улучшается в течение 15 эпох, после чего восстанавливает лучшие веса. Модель обучается на 82 изображениях с размером батча 32, максимум 50 эпох, с валидацией на 20 изображениях. После обучения выводится итоговая точность на валидационной выборке.

```
model = Sequential([
    Dense(54, input_dim=400, activation='relu'),
    Dropout(0.5),
    Dense(27, activation='relu'),
    Dropout(0.5),
    Dense(CLASS_COUNT, activation='softmax')
])

model.compile(
    loss='categorical_crossentropy',
    optimizer=Adam(learning_rate=0.001),
    metrics=['accuracy']
)

early_stop = EarlyStopping(
    monitor='val_accuracy',
    patience=15,
    restore_best_weights=True
)

history = model.fit(
    x_train_split, y_train_split,
    batch_size=32,
    epochs=50,
    validation_data=(x_val, y_val),
    callbacks=[early_stop],
    verbose=1
)

test_loss, test_acc = model.evaluate(x_val, y_val, verbose=0)
print(f"Точность: {test_acc:.4f} ({test_acc*100:.1f}%)")
```

Рисуноу 3.6 – Создание и обучение модели

```
Epoch 1/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 4s 993ms/step - accuracy: 0.4783 - loss: 0.8832 - val_accuracy: 0.7000 - val_loss: 0.5481
Epoch 2/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - accuracy: 0.6643 - loss: 0.7046 - val_accuracy: 0.5500 - val_loss: 0.6601
Epoch 3/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step - accuracy: 0.5500 - loss: 0.7589 - val_accuracy: 0.6000 - val_loss: 0.6378
Epoch 4/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - accuracy: 0.5644 - loss: 0.7507 - val_accuracy: 0.8500 - val_loss: 0.5765
Epoch 5/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step - accuracy: 0.5656 - loss: 0.7036 - val_accuracy: 0.8500 - val_loss: 0.5914
Epoch 6/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 60ms/step - accuracy: 0.5212 - loss: 0.7041 - val_accuracy: 0.7000 - val_loss: 0.6261
Epoch 7/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - accuracy: 0.5874 - loss: 0.6750 - val_accuracy: 0.7500 - val_loss: 0.6301
Epoch 8/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 59ms/step - accuracy: 0.5978 - loss: 0.6547 - val_accuracy: 0.8500 - val_loss: 0.6013
Epoch 9/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - accuracy: 0.6213 - loss: 0.6177 - val_accuracy: 0.8500 - val_loss: 0.5349
Epoch 10/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 56ms/step - accuracy: 0.5817 - loss: 0.6656 - val_accuracy: 0.8500 - val_loss: 0.5445
Epoch 11/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 58ms/step - accuracy: 0.6352 - loss: 0.5961 - val_accuracy: 0.8000 - val_loss: 0.5858
Epoch 12/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - accuracy: 0.7169 - loss: 0.6074 - val_accuracy: 0.9000 - val_loss: 0.5367
Epoch 13/50
...
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 0.8365 - loss: 0.4072 - val_accuracy: 0.9500 - val_loss: 0.2891
Epoch 33/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.7470 - loss: 0.4333 - val_accuracy: 0.9000 - val_loss: 0.2862
Точность: 0.9500 (95.0%)
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Рисунок 3.7 – Создание и обучение модели

3.7. Сохранение модели и визуализация предсказаний.

Выполняется финальная оценка модели и визуализация результатов. Обученная модель сохраняется в файл для последующего использования. Модель делает предсказания для всех изображений валидационной выборки, которые сравниваются с истинными метками. Для наглядной демонстрации случайным образом выбираются 16 изображений. Каждое изображение отображается в оттенках серого, а в заголовке указываются предсказанный класс (сверху) и истинный класс (снизу). Правильные предсказания выделяются зеленым цветом, ошибки – красным. Общий заголовок графика содержит итоговую точность модели, что позволяет визуально оценить качество ее работы.

```python
model.save('xo_model_95.keras')
```

```python
# Получаем предсказания
y_pred = model.predict(x_val, verbose=0)
y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = np.argmax(y_val, axis=1)

# Названия классов
class_names = ['X КРЕСТИК', 'o НОЛИК']

# Покажем 16 изображений
plt.figure(figsize=(16, 8))
for i in range(16):
    idx = np.random.randint(len(x_val))
    img = x_val[idx].reshape(20, 20)
    true = y_true_classes[idx]
    pred = y_pred_classes[idx]

    plt.subplot(4, 4, i+1)
    plt.imshow(img, cmap='gray')
    color = 'green' if pred == true else 'red'

    plt.title(f'П:{class_names[pred]}\nИ:{class_names[true]}',
              color=color, fontsize=9)
    plt.axis('off')

plt.suptitle(f'Предсказания модели. Точность: {test_acc*100:.1f}%', fontsize=14)
plt.tight_layout()
plt.show()
```

Рисунок 3.8 – Сохранение модели и визуализация предсказаний

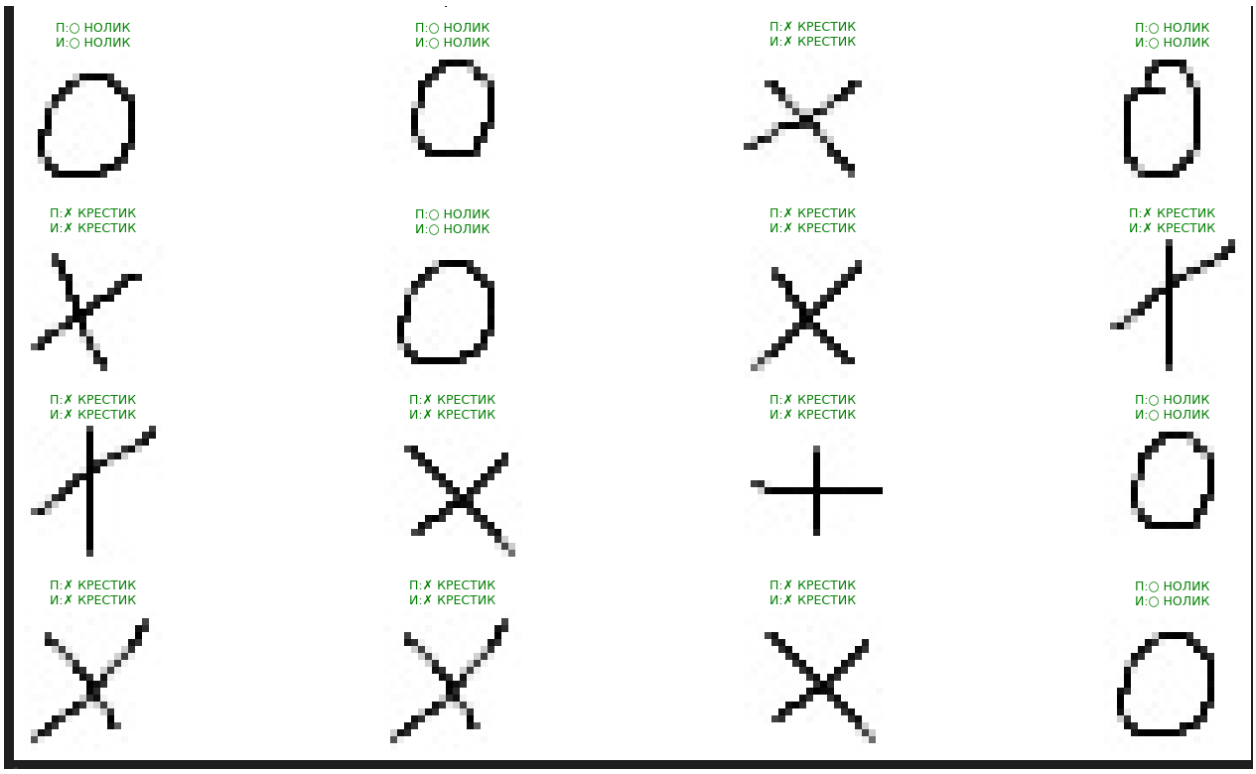

Рисунок 3.9 – Визуализация предсказаний

Задание 4. Распознайте рукописную цифру, написанную на листе от руки.

Последовательность шагов следующая:

– На бумаге рисуем произвольную цифру (желательно нарисовать цифру размером не более 5 * 5 мм и без наклона. В занятии нейронка обучалась на цифрах американских студентов. Эти цифры были написаны на тетрадных листах в клетку и имели схожий размер);

– Фотографируем. Загружаем фото в Collaboratory;

– С помощью функции image.load_img(path, target_size=(28, 28), color_mode = ‘grayscale’) загружаем картинку в переменную;

– С помощью функции image.img_to_array(img) преобразуем изображение в numpy–массив;

– Выполняем инверсию цветов, нормирование и решейп массива;

– Выполняем распознавание собственной рукописной цифры;

Примечание: точность распознавания рукописных цифр может быть достаточно низкой, т.к. рукописные цифры после преобразований хоть и

похожи на содержащиеся в базе, но могут отличаться по конфигурации, толщине линий и т.д.

4.1. Подключение необходимых библиотек.

Подключаются все необходимые библиотеки. Из TensorFlow/Keras импортируются компоненты: датасет MNIST для загрузки эталонных данных, Sequential для построения модели, Dense и Flatten для слоев нейросети, to_categorical для one–hot кодирования, модуль image для загрузки пользовательских изображений и EarlyStopping для ранней остановки. Также выводится версия TensorFlow для контроля окружения.

```
import numpy as np
import matplotlib.pyplot as plt
import os
import gdown
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.preprocessing import image
from tensorflow.keras.callbacks import EarlyStopping
import tensorflow as tf

print("Версия TensorFlow:", tf.__version__)
```

Рисунок 4.1 – Подключение необходимых библиотек

4.2. Загрузка и подготовка датасета MNIST.

Загружается классический датасет MNIST, содержащий 70000 изображений рукописных цифр размером 28x28 пикселей. Выполняется нормализация пикселей делением на 255 для приведения значений к диапазону [0, 1].

```python
# Загружаем MNIST
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Нормализация (пиксели от 0 до 1)
x_train = x_train.astype('float32') / 255
x_test = x_test.astype('float32') / 255

# One-hot encoding для меток
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)

print(f"Обучающая выборка: {x_train.shape}")
print(f"Тестовая выборка: {x_test.shape}")
```

```
Обучающая выборка: (60000, 28, 28)
Тестовая выборка: (10000, 28, 28)
```

Рисунок 4.2 – Загрузка и подготовка датасета MNIST

4.3. Создание и обучение модели на MNIST

Создается простая нейронная сеть для распознавания цифр. Первый слой Flatten преобразует двумерное изображение 28x28 в одномерный вектор длиной 784. Затем идет полносвязный слой со 128 нейронами и активацией ReLU. Выходной слой содержит 10 нейронов (по числу цифр) с активацией softmax, дающей вероятности принадлежности к каждому классу. Модель компилируется с оптимизатором adam, функцией потерь categorical_crossentropy и метрикой точности.

Используется EarlyStopping с отслеживанием val_loss: если ошибка на валидации не уменьшается в течение 3 эпох, обучение останавливается и восстанавливаются лучшие веса. Модель обучается на 80% данных (48000 изображений), 20% (12000) используются для валидации. После обучения проверяется точность на официальном тестовом наборе MNIST (10000 изображений).

```python
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

# Ранняя остановка для предотвращения переобучения
early_stop = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)

# Обучение
history = model.fit(
    x_train, y_train_cat,
    batch_size=128,
    epochs=20,
    validation_split=0.2,
    callbacks=[early_stop],
    verbose=1
)

# Проверка на официальном тесте MNIST
test_loss, test_acc = model.evaluate(x_test, y_test_cat, verbose=0)
print(f"\nТочность на тесте MNIST: {test_acc:.4f} ({test_acc*100:.1f}%)")
```

Рисунок 4.3 – Создание и обучение модели на MNIST

```
Epoch 1/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 8ms/step - accuracy: 0.8071 - loss: 0.6999 - val_accuracy: 0.9435 - val_loss: 0.2076
Epoch 2/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9446 - loss: 0.1964 - val_accuracy: 0.9569 - val_loss: 0.1564
Epoch 3/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9601 - loss: 0.1405 - val_accuracy: 0.9625 - val_loss: 0.1304
Epoch 4/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9690 - loss: 0.1057 - val_accuracy: 0.9667 - val_loss: 0.1149
Epoch 5/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 4s 10ms/step - accuracy: 0.9761 - loss: 0.0852 - val_accuracy: 0.9690 - val_loss: 0.1068
Epoch 6/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.9804 - loss: 0.0694 - val_accuracy: 0.9701 - val_loss: 0.1000
Epoch 7/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9829 - loss: 0.0598 - val_accuracy: 0.9717 - val_loss: 0.0936
Epoch 8/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9871 - loss: 0.0469 - val_accuracy: 0.9728 - val_loss: 0.0928
Epoch 9/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9886 - loss: 0.0425 - val_accuracy: 0.9726 - val_loss: 0.0897
Epoch 10/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9911 - loss: 0.0359 - val_accuracy: 0.9717 - val_loss: 0.0941
Epoch 11/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.9922 - loss: 0.0294 - val_accuracy: 0.9739 - val_loss: 0.0945
Epoch 12/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 3s 8ms/step - accuracy: 0.9925 - loss: 0.0300 - val_accuracy: 0.9732 - val_loss: 0.0894
Epoch 13/20
...
Epoch 16/20
375/375 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.9978 - loss: 0.0128 - val_accuracy: 0.9740 - val_loss: 0.0950

Точность на тесте MNIST: 0.9775 (97.8%)
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Рисунок 4.4 – Создание и обучение модели на MNIST

4.4. Загрузка собственных изображений цифр.

С помощью gdown загружается архив с изображениями цифр, нарисованных в Paint. Архив распаковывается в директорию num_paint. Внутри находятся 10 изображений (0.jpg – 9.jpg), каждое содержит соответствующую цифру, нарисованную вручную. Эти изображения будут использованы для тестирования модели на данных, похожих на реальное рукописное письмо.

```
[52]
url = 'https://drive.google.com/uc?id=1VtKeGdn6PdOMl245sWSbVl39otp65XC-'
gdown.download(url, 'num.zip', quiet=False)
!unzip -o -q num.zip -d num_paint

print("Загруженные файлы:")
!ls num_paint

Downloading...
From: https://drive.google.com/uc?id=1VtKeGdn6PdOMl245sWSbVl39otp65XC-
To: /content/num.zip
100%|██████████| 36.4k/36.4k [00:00<00:00, 24.5MB/s]
Загруженные файлы:
0.jpg  1.jpg  2.jpg  3.jpg  4.jpg  5.jpg  6.jpg  7.jpg  8.jpg  9.jpg
```

Рисунок 4.5 – Загрузка собственных изображений цифр

4.5. Функция подготовки изображений.

Создается функция для подготовки пользовательских изображений к подаче в модель, обученную на MNIST. Изображение загружается, изменяется размер до 28x28 пикселей и конвертируется в оттенки серого. Важный шаг – инверсия цветов (255 – img_array), так как в MNIST цифры белые на черном фоне, а в Paint обычно рисуют черным на белом. Затем выполняется нормализация делением на 255. В конце убирается канал, чтобы изображение имело форму (28, 28), как в MNIST.

```python
def load_and_prepare(image_path):
    """Загружает изображение, подготавливает его для модели."""
    img = image.load_img(image_path, target_size=(28, 28), color_mode='grayscale')
    img_array = image.img_to_array(img)

    # Инверсия, так как MNIST ожидает белую цифру на черном фоне
    img_array = 255 - img_array

    # Нормализация
    img_array = img_array / 255.0

    img_array = img_array.reshape(28, 28)

    return img_array
```

Рисунок 4.6 – Функция подготовки изображений

4.6. Тестирование модели на своих изображениях.

В этом блоке выполняется тестирование обученной модели на 10 собственных изображениях цифр. Для каждой цифры от 0 до 9 загружается соответствующий файл, изображение подготавливается с помощью функции load_and_prepare. Поскольку модель ожидает на входе батч изображений, к изображению добавляется измерение (1, 28, 28). Модель делает предсказание, из которого извлекается наиболее вероятная цифра (argmax) и значение уверенности. Результаты сохраняются в список вместе с истинной меткой и флагом правильности предсказания. Изображения сохраняются для последующей визуализации.

```python
image_dir = 'num_paint'
results = []
images_for_display = []
filenames = []

for i in range(10):
    filename = f"{i}.jpg"
    img_path = os.path.join(image_dir, filename)

    if os.path.exists(img_path):
        # Загружаем и подготавливаем изображение
        img_processed = load_and_prepare(img_path)

        # Модель ожидает батч изображений, поэтому добавляем измерение
        img_batch = img_processed.reshape(1, 28, 28)

        # Предсказание
        prediction = model.predict(img_batch, verbose=0)[0]
        predicted_digit = np.argmax(prediction)
        confidence = prediction[predicted_digit]

        results.append({
            'Истина': i,
            'Предсказано': predicted_digit,
            'Уверенность': confidence,
            'Верно': (i == predicted_digit)
        })
        images_for_display.append(img_processed)
        filenames.append(filename)
    else:
        print(f"Файл {filename} не найден!")

print("Предсказания получены.")
```

Рисунок 4.7 – Тестирование модели на своих изображениях

4.7. Визуализация результатов.

```python
plt.figure(figsize=(15, 8))
for idx, (img, res) in enumerate(zip(images_for_display, results)):
    plt.subplot(2, 5, idx+1)
    plt.imshow(img, cmap='gray')
    color = 'green' if res['Верно'] else 'red'
    plt.title(f"{filenames[idx]}\nП: {res['Предсказано']} (И: {res['Истина']})\n{res['Уверенность']:.2f}",
              color=color, fontsize=10)
    plt.axis('off')
plt.suptitle('Результаты распознавания цифр, нарисованных в Paint', fontsize=14)
plt.tight_layout()
plt.show()
```

Рисунок 4.8 – Визуализация результатов

Рисунок 4.9 – Визуализация результатов

Вывод: в ходе выполнения лабораторной работы были изучены основы архитектуры простых нейронных сетей и получены практические навыки их программной реализации на Python с использованием фреймворков TensorFlow и PyTorch. В процессе работы были рассмотрены и экспериментально опробованы различные функции активации, оптимизаторы и функции потерь, что позволило понять их влияние на процесс обучения моделей. В результате были приобретены навыки подбора гиперпараметров для эффективного решения задач анализа данных, что подтверждает успешное достижение поставленной цели.